编译系统

第三章

词法分析

哈尔滨工业大学 陈鄞

第3讲（词法分析）要点

- 词法分析的任务：识别单词
  - 单词的描述
  - 单词的识别

主观题 2分

我们可以通过哪些方式来描述单词?

1、单词的描述

- RG（正则文法）：G
  - 正则文法G生成的语言：$L(G)$

例：标识符

① $S \rightarrow a \mid b \mid c \mid d$

② $S \rightarrow aT \mid bT \mid cT \mid dT$

③ $T \rightarrow a \mid b \mid c \mid d \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5$

④ $T \rightarrow aT \mid bT \mid cT \mid dT \mid 0T \mid 1T \mid 2T \mid 3T \mid 4T \mid 5T$

- RE（正则表达式）：r 更直观、更紧凑
  - 正则表达式r表示的语言：$L(r)$

例：标识符

$(a \mid b \mid c \mid d)(a \mid b \mid c \mid d \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5)^*$

- RD（正则定义）：$d \rightarrow r$ 给RE命名

正则定义（*Regular Definition*）

- 正则定义是具有如下形式的定义序列：

$$d_1 \rightarrow r_1$$
$$d_2 \rightarrow r_2$$
$$\dots$$
$$d_n \rightarrow r_n$$

给一些*RE*命名，并在之后的*RE*中像使用字母表中的符号一样使用这些名字

其中：

- 每个d_i都是一个新符号，它们都不在字母表Σ中，而且各不相同
- 每个r_i是字母表$\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$上的正则表达式

例1

- C语言中标识符的正则定义
  - $digit \rightarrow 0|1|2|\ldots|9$
  - $letter_ \rightarrow A|B|\ldots|Z|a|b|\ldots|z|_$
  - $id \rightarrow letter_(letter_|digit)^*$

例2

- （整型或浮点型）无符号数的正则定义
  - $digit \rightarrow 0|1|2|\ldots|9$
  - $digits \rightarrow digit\ digit^*$
  - $optionalFraction \rightarrow .digits|\varepsilon$
  - $optionalExponent \rightarrow (\ E(+|-|\varepsilon)digits\ )|\varepsilon$
  - $number \rightarrow digits\ \ optionalFraction\ \ optionalExponent$

例：	2	2.15	2.15*E*+3	2.15*E*-3	2.15*E*3	2*E*-3

2、单词的识别

- 理论基础：有穷自动机（FA）

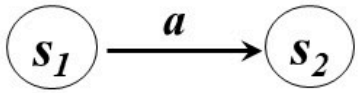
$$M=(S,\ \Sigma,\ \delta,\ s_0,\ F)$$

  - S：状态集合
  - Σ：输入符号集合
  - δ：转换函数，$S\times\Sigma\rightarrow 2^S$
  - s_0：初始状态，$s_0\in S$
  - F：终止状态集合，$F\subseteq S$

①FA的表示

$$M=(S,\ \Sigma,\ \delta,\ s_0,\ F)$$

- 转换图
  - 节点　　状态
  - 边（有向）　输入符号
- 转换表



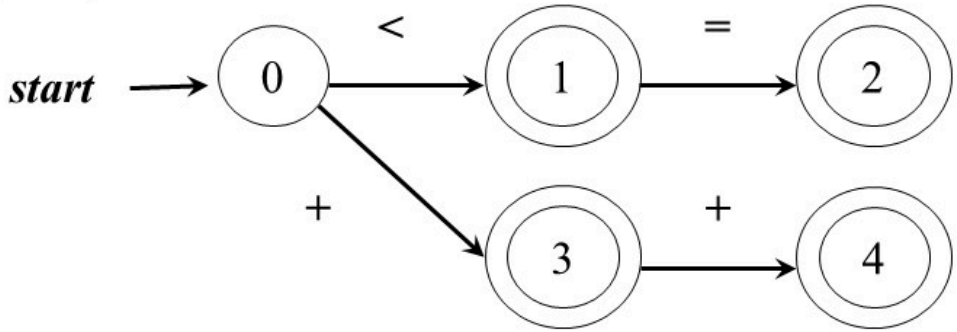
S \ Σ	a_1	a_2	…	a_n
s_1				
s_2				
…				
s_m				

②FA接受（识别）的语言

- 由一个有穷自动机M接收的所有串构成的集合称为是该FA接受（或识别）的语言，记为$L(M)$
  - 给定输入串x，如果存在一个对应于串x的从初始状态到某个终止状态的转换序列，则称串x被该FA接受

②FA接受（识别）的语言

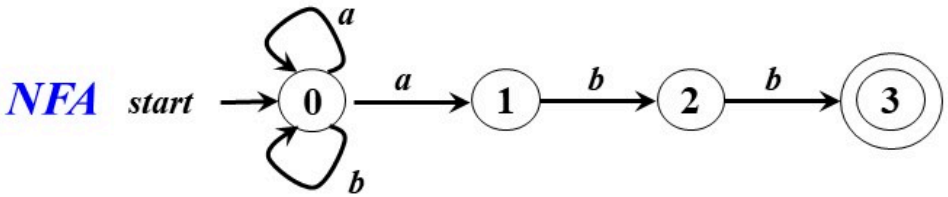
- 最长子串匹配原则
  - 当输入串的多个前缀与一个或多个模式匹配时，总是选择最长的前缀进行匹配

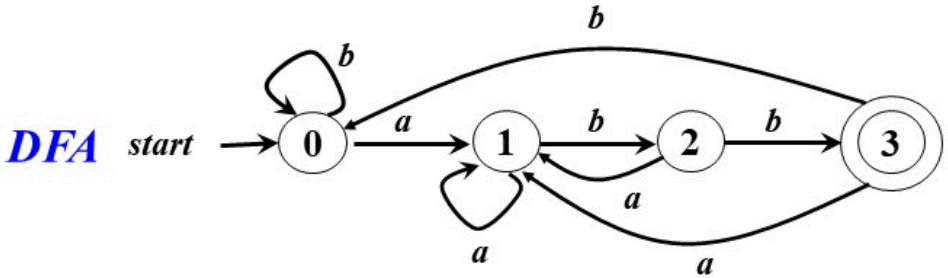


  - 在到达某个终态之后，只要输入带上还有符号，DFA就继续前进，以便寻找尽可能长的匹配

③FA的分类　　$M=(S,\ \Sigma,\ \delta,\ s_0,\ F)$

- *DFA*（确定的有穷自动机）　更适用
  - δ: $S\times\Sigma\rightarrow S$
- *NFA*（非确定的有穷自动机）更直观
  - δ: $S\times\Sigma\rightarrow 2^S$



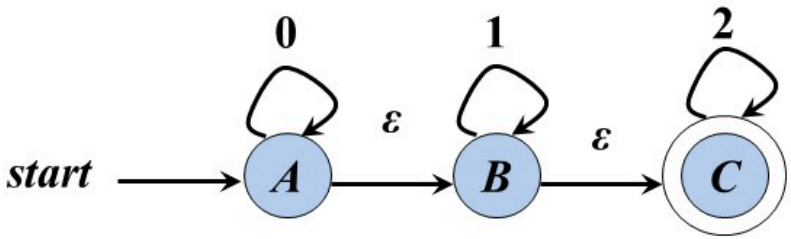


③FA的分类

$M=(S,\ \Sigma,\ \delta,\ s_0,\ F)$

- *DFA*（确定的有穷自动机）
  - δ: $S\times\Sigma\rightarrow S$

- *NFA*（非确定的有穷自动机）
  - δ: $S\times\Sigma\rightarrow 2^S$ 不带“ε-边”
  - δ: $S\times(\Sigma\cup\{\varepsilon\})\rightarrow 2^S$ 带“ε-边”

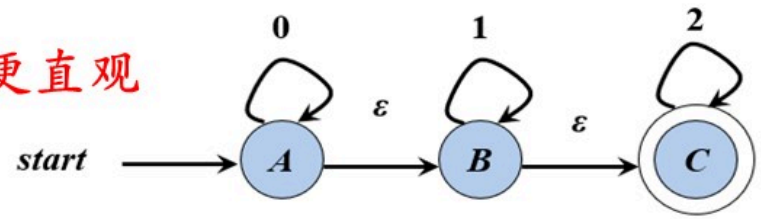


$r=0^*1^*2^*$

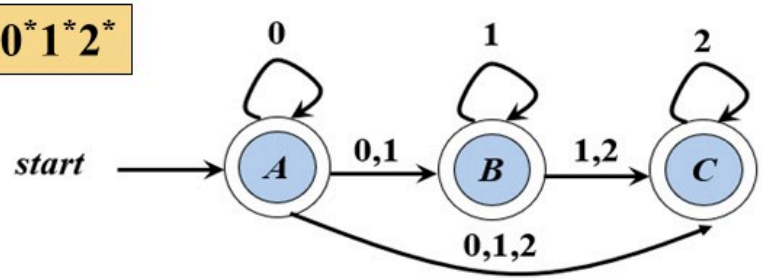
③FA的分类

$M=(S,\ \Sigma,\ \delta,\ s_0,\ F)$

- *DFA*（确定的有穷自动机）
  - δ: $S\times\Sigma\rightarrow S$
- *NFA*（非确定的有穷自动机）
  - δ: $S\times\Sigma\rightarrow 2^S$ 不带“ε-边”
  - δ: $S\times(\Sigma\cup\{\varepsilon\})\rightarrow 2^S$ 带“ε-边”更直观



$r=0^*1^*2^*$



④DFA的实现 $M=(S, \Sigma, \delta, s_0, F)$

➢ 函数体由一个循环构成

```
s = s0;
c = nextChar();
while (c! = eof) {
        s = move(s, c);
        c = nextChar();
}
if (s在F中) return "yes";
else return "no";
```

表驱动

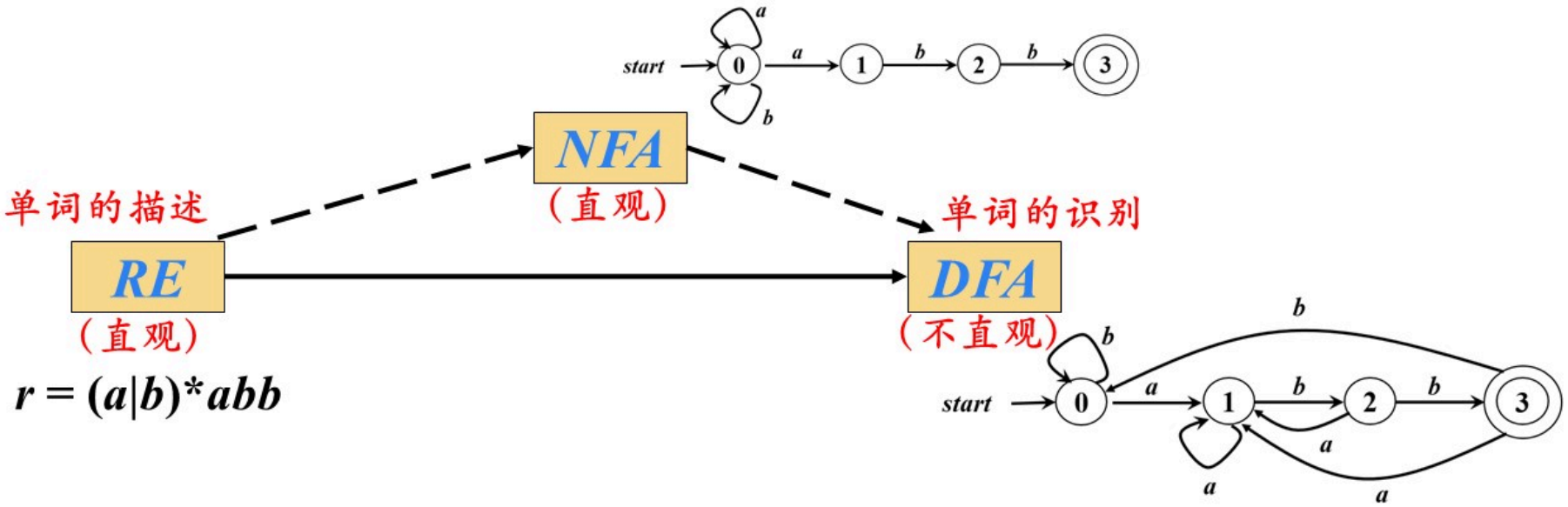
s \ Σ	a_1	a_1	…	a_n
s_1				
s_2				
…				
s_m				

⑤FA与RE的等价性

- $RE \Leftrightarrow FA$

  $r \Leftrightarrow M$

- 词法分析器（*scanner*）的实现思路

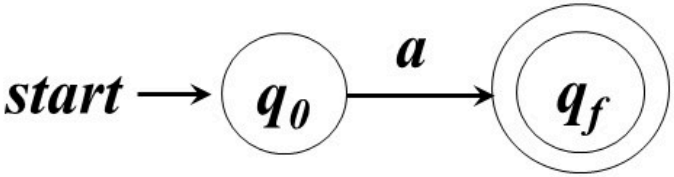


⑥从RE 到NFA

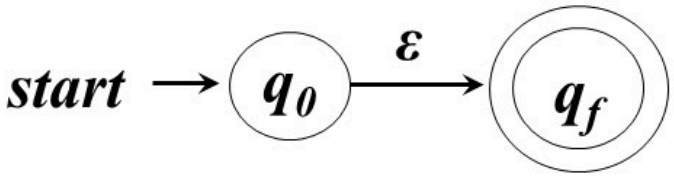
Ken Thompson(1968). "Regular expression search algorithm（正则表达式搜索算法）"

肯·汤普森
1983年图灵奖得主
UNIX 之父
C 语言的前身 B 语言的开发者
Go 语言联合创始人

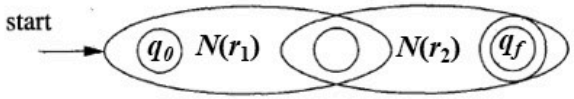
➢字母表Σ中符号*a*对应的*NFA*



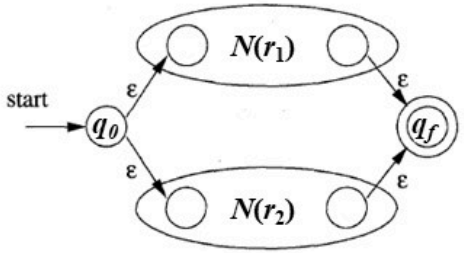
➢ε对应的*NFA*



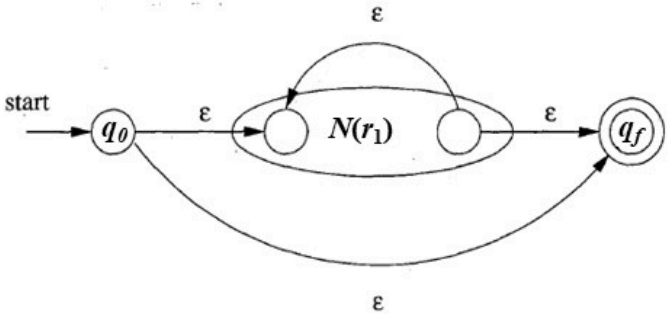
- 假设正则表达式r_1和r_2对应的NFA分别为$N(r_1)$和$N(r_2)$
  - $r = r_1 r_2$对应的NFA



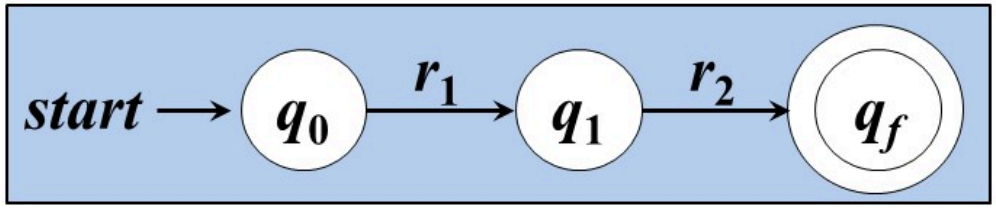
  - $r = r_1 | r_2$对应的NFA



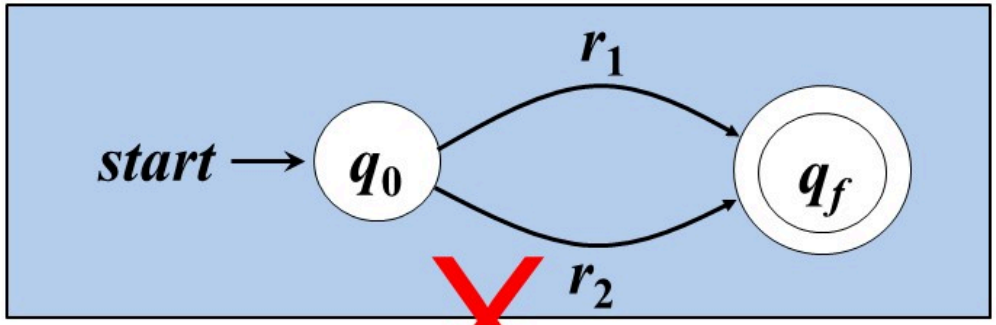
  - $r = (r_1)^*$对应的NFA



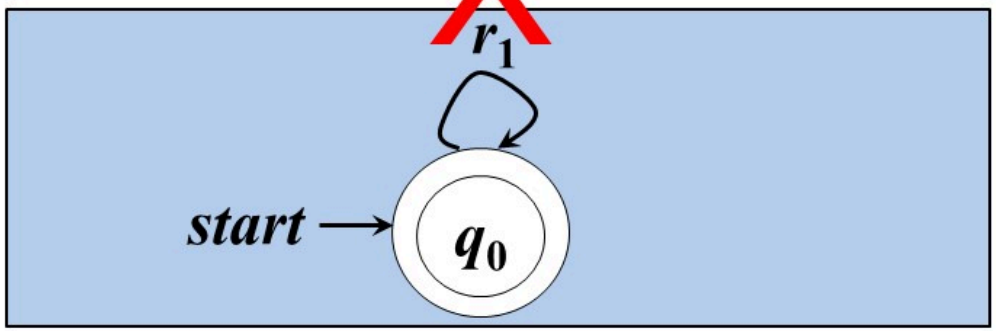
- $r = r_1 r_2$ 对应的 NFA



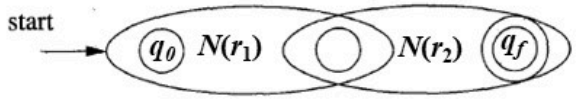
- $r = r_1 | r_2$ 对应的 NFA



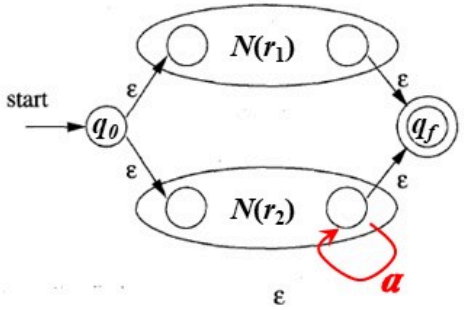
- $r = (r_1)^*$ 对应的 NFA



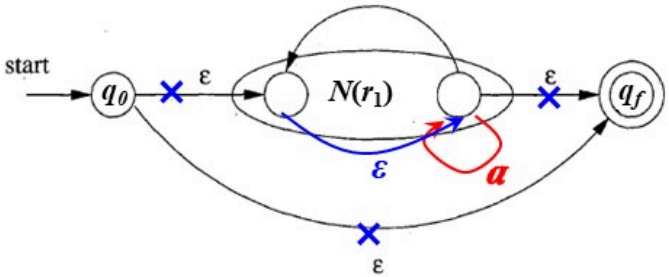
- 假设正则表达式r_1和r_2对应的NFA分别为$N(r_1)$和$N(r_2)$
  - $r = r_1 r_2$对应的NFA



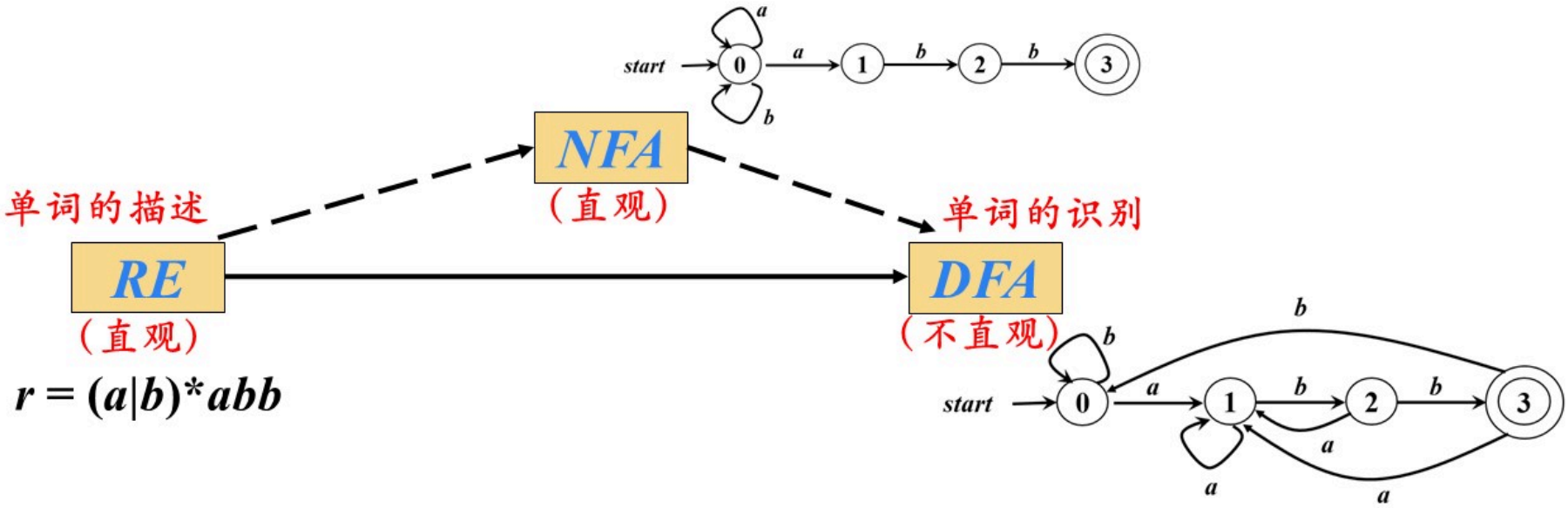
  - $r = r_1 | r_2$对应的NFA



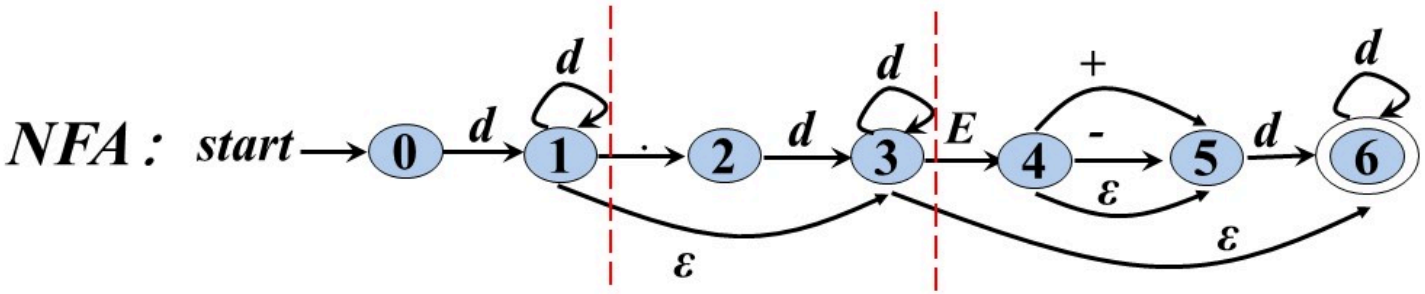
  - $r = (r_1)^*$对应的NFA



⑦从NFA到DFA：子集构造法

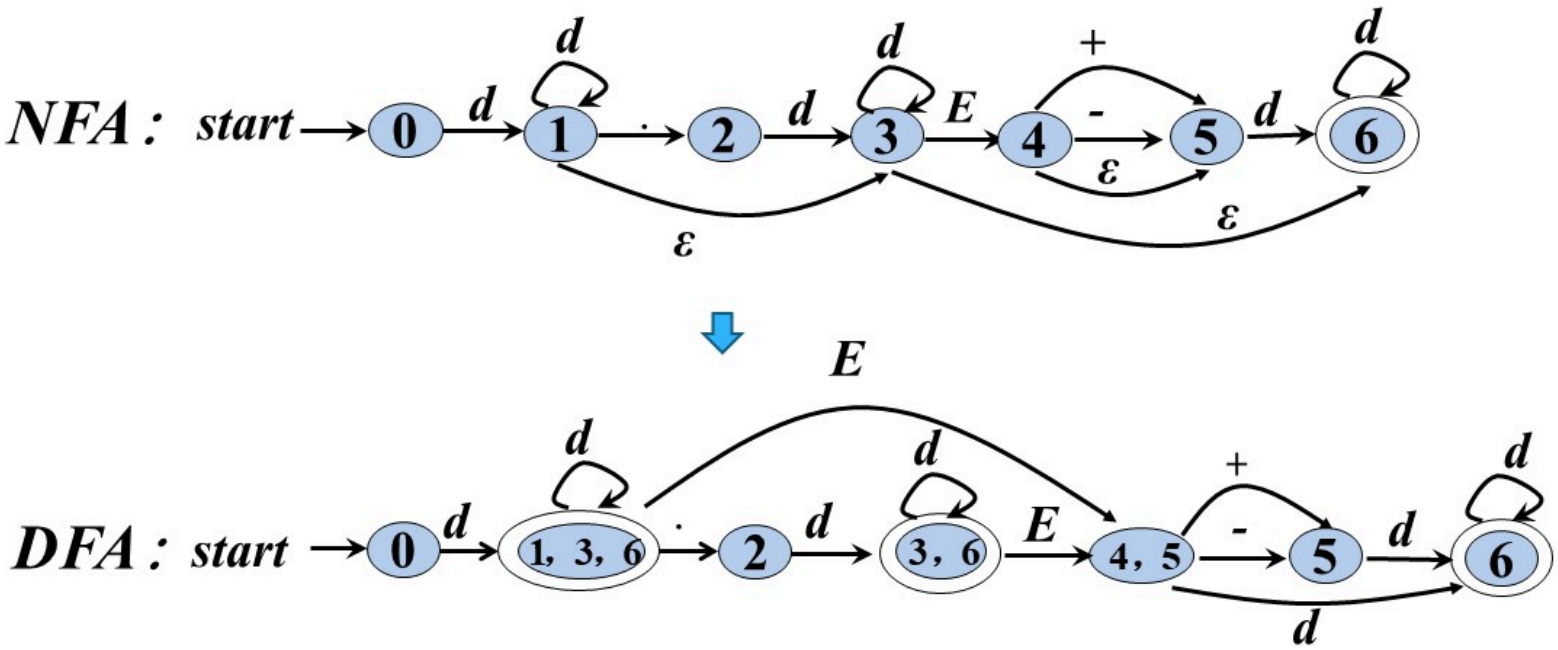


例



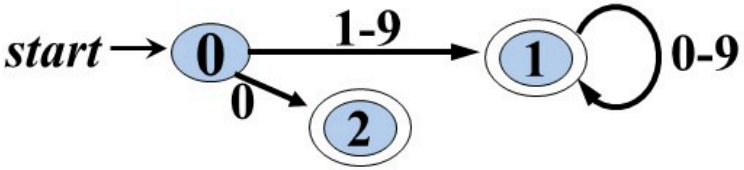
- *digit* → 0|1|2|…|9
- *digits* → *digit digit**
- *optionalFraction* → .*digits*|ε
- *optionalExponent* → (*E*(+|-|ε)*digits*)|ε
- *number* → *digits optionalFraction optionalExponent*

例

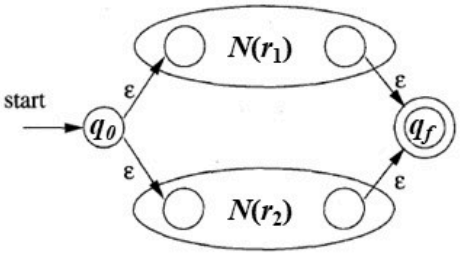


识别各进制无符号整数的*DFA*

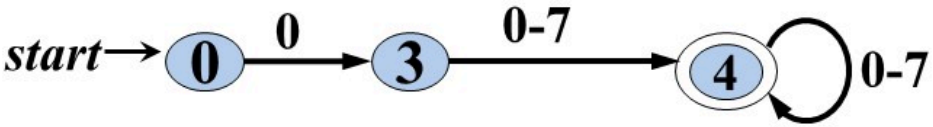
$DEC \rightarrow (1|...|9)(0|...|9)^*|0$



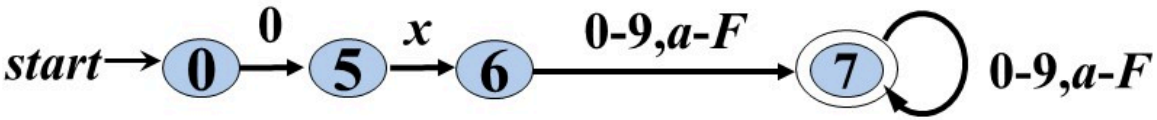
$number \rightarrow DEC \mid OCT \mid HEX$



$OCT \rightarrow 0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$

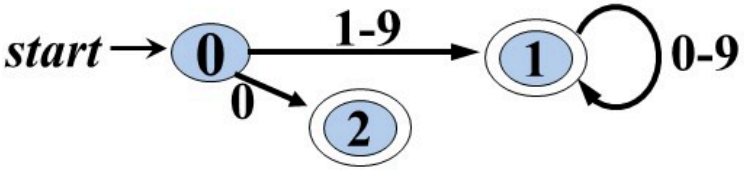


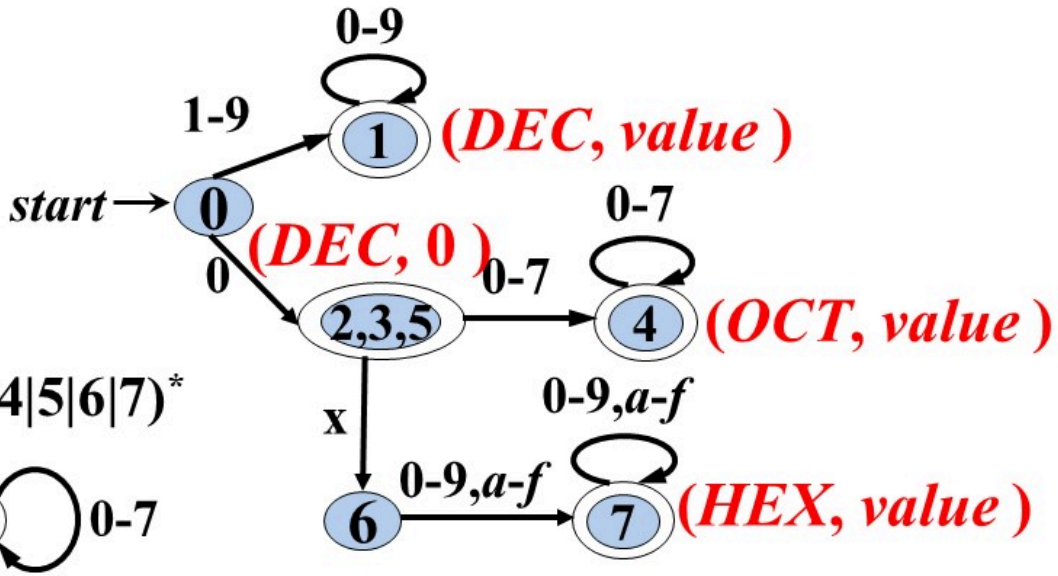
$HEX \rightarrow 0x(0|1|...|9|a|...|f|A|\ldots|F)(0|...|9|a|...|f|A|\ldots|F)^*$



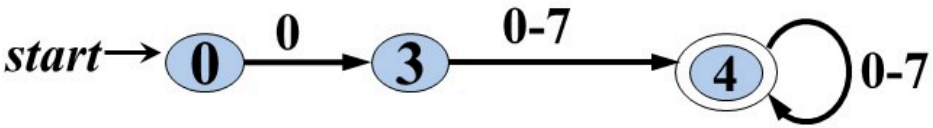
识别各进制无符号整数的*DFA*

$DEC \rightarrow (1|...|9)(0|...|9)^*|0$

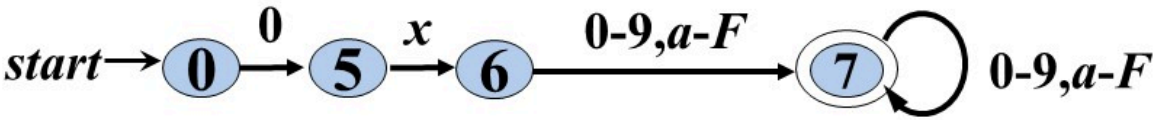




$OCT \rightarrow 0(0|1|2|3|4|5|6|7)(0|1|2|3|4|5|6|7)^*$



$HEX \rightarrow 0x(0|1|...|9|a|...|f|A|\ldots|F)(0|...|9|a|...|f|A|\ldots|F)^*$



提纲

3.1 单词的描述

3.2 单词的识别

3.3 词法分析阶段的错误处理

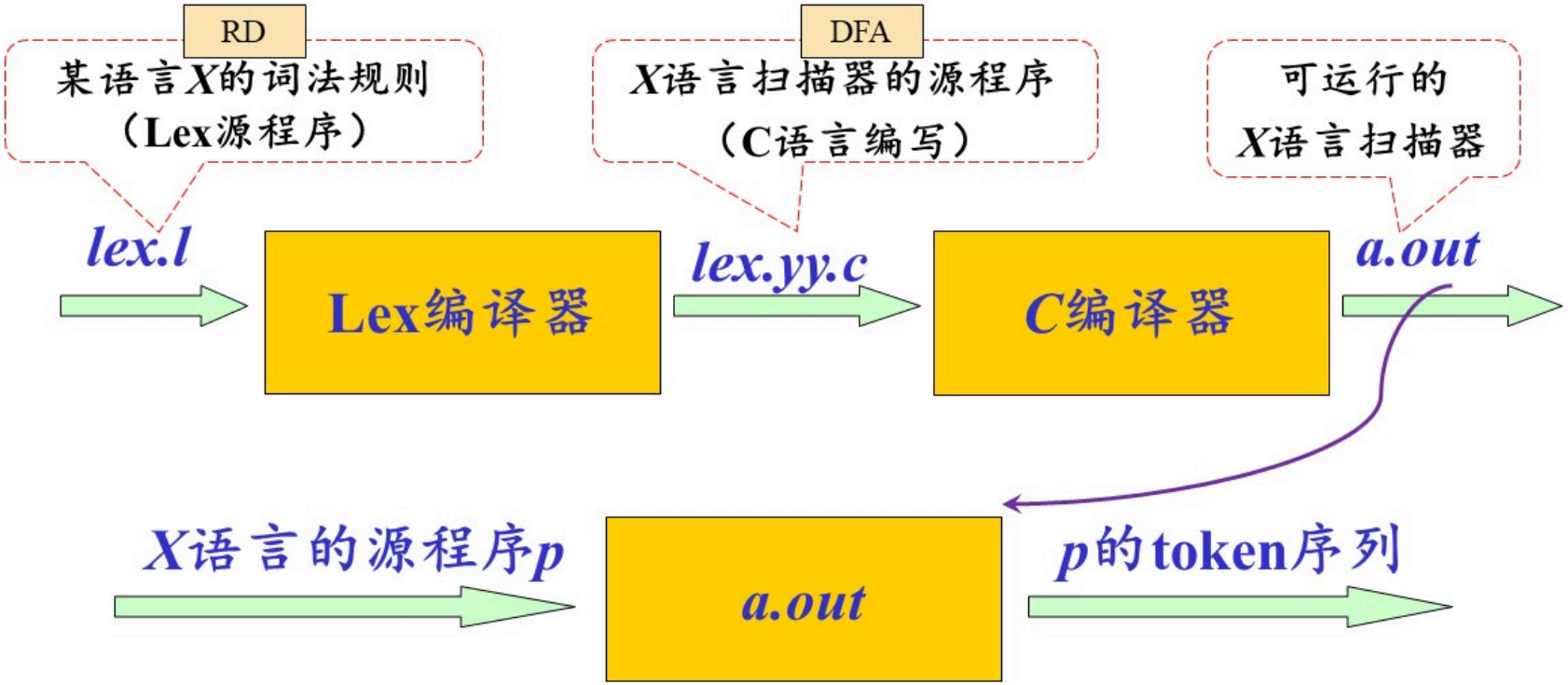
3.4 词法分析器生成工具Lex

3.4 词法分析器生成工具Lex

Lesk, M.E. (October 1975). "Lex – A Lexical Analyzer Generator". *Comp. Sci. Tech. Rep. No. 39* (Murray Hill, New Jersey: Bell Laboratories).

- Lex的构成
  - Lex语言：用于描述各类单词的正则定义式，及各单词被识别出来以后应执行的动作
  - Lex编译器：将输入的lex源程序（RD）转化成一个词法分析器（DFA）

Lex的使用


RD
某语言X的词法规则
（Lex源程序）
DFA
X语言扫描器的源程序
（C语言编写）
可运行的
X语言扫描器
lex.l
Lex编译器
lex.yy.c
C编译器
a.out
X语言的源程序p
a.out
p的token序列

Lex程序的结构

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
%}

/* regular definitions */
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%

{ws}        {/* no action and no return */}
if          {return(IF);}
then        {return(THEN);}
else        {return(ELSE);}
{id}        {yylval = (int) installID(); return(ID);}
{number}    {yylval = (int) installNum(); return(NUMBER);}
"<"         {yylval = LT; return(RELOP);}
"<="        {yylval = LE; return(RELOP);}
"="         {yylval = EQ; return(RELOP);}
"<>"        {yylval = NE; return(RELOP);}
">"         {yylval = GT; return(RELOP);}
">="        {yylval = GE; return(RELOP);}

%%

int installID() {/* function to install the lexeme, whose
                    first character is pointed to by yytext,
                    and whose length is yyleng, into the
                    symbol table and return a pointer
                    thereto */

}

int installNum() {/* similar to installID, but puts numer-
                     ical constants into a separate table */
}
```

出现在括号内的所有内容都被直接复制到文件lex.yy.c中

声明部分

转换规则

模式 {动作}

出现在辅助部分中的所有内容都被直接复制到文件lex.yy.c中

辅助过程

扫描器自动生成的意义

- Lex的构成加快了分析器的实现速度
  - 程序员只需在很高的模式层次上描述软件，就可以依赖自动生成工具来生成详细的代码
- 修改扫描器的工作变得更加简单
  - 只需修改那些受到影响的模式，无需改写整个程序

种别码编码方案举例：龙书附录A.3

```
package lexer;                    // File Tag.java
public class Tag {
   public final static int
      AND   = 256,  BASIC = 257,  BREAK = 258,  DO    = 259,  ELSE  = 260,
      EQ    = 261,  FALSE = 262,  GE    = 263,  ID    = 264,  IF    = 265,
      INDEX = 266,  LE    = 267,  MINUS = 268,  NE    = 269,  NUM   = 270,
      OR    = 271,  REAL  = 272,  TEMP  = 273,  TRUE  = 274,  WHILE = 275;
}
```

- 关键字
  - BASIC: int/float/char/bool
  - BREAK: beak
  - DO: do
  - WHILE: while
  - IF: if
  - ELSE: else
  - TRUE: true
  - FALSE: false
- 算符（非单符）
  - AND
  - OR
  - EQ: ==
  - NE: !=
  - GE: >=
  - LE: <=
- 常量
  - NUM整型
  - REAL浮点型
- 标识符
  - ID

单字符词法单元就用其自身的ASCII码作为种别码

用大于255的整数来表示非单字符词法单元的种别码

INDEX、MINUS、TEMP不是词法单元，它们将在抽象语法树中使用

结束